Bimetal: One Rust API for Python, Node.js and the Browser

Felipe Faria

ABSTRACT

Bimetal turns one Rust library into a Python package and a TypeScript package that call the same compiled code. Today, an author who wants both packages writes a binding layer for each language, and then keeps the types, the validation and the documentation of the two layers in agreement by hand. Bimetal instead reads the API from the compiled library: marked Rust items register a description of themselves when they are compiled, and a command loads a build of the library and asks it for that description. From the description, generators write Pydantic models, TypeScript declarations and a class for each Rust value that stays alive between calls. The generated files are committed, and a check fails when a fresh build produces different bytes. Because Python, WebAssembly and Node.js differ in what a call may do, this paper also states what each one promises when another call is using a value, when a call is cancelled and when Rust panics. On a laptop, a call with no arguments takes 53 ns in Python, and a batch of a million Arrow rows passes from Python into Rust and back in 4.8 μs.

KEYWORDS

language bindings, foreign function interfaces, code generation, WebAssembly, reproducible builds

1 INTRODUCTION

A useful library written in Rust eventually needs callers that do not write Rust. The usual answer is a binding layer for each language: PyO3 [18] for Python, napi-rs [14] for Node.js and wasm-bindgen [23] for the browser. Each of these tools does its job well, but each asks the author to describe the API again for that language alone. The Python types then need stubs, which PyO3's guide suggests maintaining by hand [19], the data types need validation in each language, and the documentation needs copying. The author ends up with three descriptions of one API and nothing that keeps them in agreement.

Generating bindings from one definition is an old idea, from SWIG [3] and Thrift [20] to UniFFI [13] and Diplomat [4]. Doing it for Python and the browser together is hard for two reasons. The first is that the runtimes differ in what a call may do. A Python call can block its thread on a lock, whereas JavaScript has a single thread that cannot wait for itself. Python can share Arrow memory with Rust, but JavaScript cannot, because a view into WebAssembly memory is detached whenever that memory grows. A native library can catch a panic, while in WebAssembly a panic is a trap that leaves the module's memory in an unknown state [8]. The second reason is that generated code is useful only while it matches the Rust code. A separate interface file can drift from the Rust code, and so can a parser that reads Rust source, because a `cfg` attribute, a macro or a build script can change what the compiler builds.

Existing tools avoid these problems by narrowing their scope. PyO3, napi-rs and wasm-bindgen give full control of one runtime, and the author repeats the work for the next one. UniFFI and Diplomat generate bindings for many languages, but in each language they support fewer types and language features than the single-language tools do.

Bimetal starts from one observation: a compiled library already contains an exact description of what the compiler built. Each marked item registers a description of itself when it is compiled. To read the API, a command builds the library for the current machine, loads the build, and calls a function that returns a JSON description, which we call the *contract*. Generators write a package for each language from the contract, and the author commits these packages to the repository. A change to the API therefore appears in code review, and a later build can compare the committed files with what the Rust code would generate.

This design makes three trade-offs. First, Bimetal generates code for two languages only, because each language needs its own generator, its own runtime and its own implementation of every feature. Second, a call passes through generated code that validates and converts its values, which costs time that a hand-written binding does not spend (Section 12). Third, where the runtimes differ, the packages differ too: a panic ends a WebAssembly module, and a synchronous method on a value that another call is using throws in JavaScript where it would wait in Python.

This paper makes the following contributions:

- A way to read an API from compiled Rust through a small C interface, and a set of checks that each compiled artifact agrees with what was read (Section 4).
- One function that plans the native code for three runtimes, which decodes each value once, and a count of the copies each runtime makes (Sections 5 and 6).
- A design for stateful values that keeps Rust's aliasing rule across the language boundary, with a statement of what each runtime does when another call is using a value (Section 7).
- A guard that makes every pending and later call fail with an error after a WebAssembly trap, where the trap alone would leave promises pending for ever (Section 8).
- The rules that make a committed WebAssembly module reproducible across checkouts, and a division of a build's inputs that keeps a committed fingerprint equal on every machine (Section 10).

2 BACKGROUND AND MODEL

2.1 What the Hosts Offer

Bimetal runs Rust code inside another language's runtime in three ways, each of which we call a *host*. The *extension* is a CPython extension built on PyO3, and it serves Python. The *module* is a WebAssembly module built on wasm-bindgen, and it serves TypeScript in Node.js and in browsers. The *addon* is a Node.js addon built on Node-API [15], and it serves TypeScript in Node.js alone. We refer to the extension and the addon together as the *native hosts*, and to the module and the addon as the *JavaScript hosts*. An application enables the extension, one of the JavaScript hosts, or both.

Table 1: The three hosts, and what each one offers.

	Extension	Module	Addon
Compiled file	`.abi3.so`	`.wasm`	`.node`
One build serves	A platform	All platforms	A platform
Runs in browsers	No	Yes	No
Threads in Rust	Yes	No	Yes
Async calls that wait run on	Tokio	Event loop	Tokio
A sync call can wait	Yes	No	No
Shares Arrow memory	Yes	No	No
Usable after a panic	Yes	No	Yes

Table 1 compares the three hosts, and two of its rows shape most of the design. A host that cannot block a thread cannot make a synchronous call wait for a lock, and a host that cannot share memory with Rust must copy every large value.

2.2 Assumptions

An *application* is one Cargo package with a file called `bimetal.toml` beside it. The promises that this paper makes about an application depend on five assumptions.

- **A1, build inputs.** Cargo's dependency files list every file that a build reads, and they name every environment variable that a build reads.
- **A2, a deterministic compiler.** The same toolchain, sources and flags produce the same bytes.
- **A3, one API on every target.** An exported item has the same signature and the same types on every compilation target.
- **A4, runtimes as documented.** CPython continues to honour its stable interface [22], Node.js continues to honour Node-API, and growing a WebAssembly memory detaches its views.
- **A5, one trust domain.** The caller and the Rust code form one program. Validation catches mistakes, and it does not defend against a hostile caller.

The first three assumptions can fail in practice, and each failure has a remedy. A1 fails when a build script or a procedural macro reads an input without telling Cargo, in which case a build can report that nothing changed when something did, and `bimetal build --force` recovers. A2 is at risk because local paths are among a build's inputs and differ between machines. Section 10 describes how the command removes them, and if A2 still fails, `bimetal check --rebuild` reports the difference. An author can break A3 with a `cfg_attr` that renames a field on one target, and a call on that target can then fail when it decodes a value. Section 4 describes what the command checks in place of A3.

2.3 Goals and Non-Goals

The design has four goals, in priority order: generated code that cannot silently differ from the Rust code, an API that follows each language's conventions, failures that say what happened, and calls that pay only for the checks they need.

Four things are outside the design. Bimetal does not support other languages. An author cannot shape a class differently in each language, because one signature has to mean the same thing everywhere. A call is not as cheap as a hand-written one. Finally, Rust is not isolated from its caller, so a stack overflow in Rust ends the process on a native host.

3 OVERVIEW

An author marks the items of a Rust library that callers may use. We say that a value *crosses* when it moves between Rust and Python or JavaScript, and we call the point where this happens the *boundary*. Two kinds of item cross. A *model* is a type that crosses as data, and it derives `Model` together with Serde's `Serialize` and `Deserialize`. A *resource* is a Rust value that stays alive between calls, and its `impl` block takes the attribute `#[bimetal::export]`, as does a free function. The rest of this paper uses one running example, a small library called `greeter`:

```
#[derive(Serialize, Deserialize, Model)]
pub struct GreetingRequest {
    #[bimetal(min_length = 1, max_length = 64)]
    pub name: String,
    pub tone: Tone,
}

#[bimetal::export]
pub fn greet(request: GreetingRequest)
    -> Result<Greeting, GreeterError> { ... }

#[bimetal::export]
impl Guestbook {
    #[bimetal(constructor)]
    pub fn new(event: String) -> Self { ... }
    pub fn sign(&mut self, name: String)
        -> Result<(), GreeterError> { ... }
}
```

The attribute `min_length` is a *field rule*: a limit on a model's field that every host enforces. A Python caller writes `greet(GreetingRequest(name="Ada", tone=Tone.Formal))` and receives an immutable Pydantic model [16], while a TypeScript caller writes `greet({name: "Ada", tone: Tone.Formal})` and receives a frozen object with declared types. In both languages, `Guestbook` is a class whose methods run on one Rust value. If `sign` rejects a name that contains only spaces, the caller receives an error with the code `blank_name`.

Figure 1 shows the build that produces these packages. The command `bimetal build` first writes a small library, the *link crate*, which depends on the application. It then compiles the link crate once for each enabled host, and once more as a *discovery build*, which yields the contract. At most three compilations run, and they run at the same time. Two generators, one for each language, then write the generated code, which we call a *package*. Each package goes into a *project*, which is an ordinary Python or npm project that holds the author's manifest and tests. Finally, the command installs the compiled extension and addon. These two files are not committed, because each platform builds its own.

Four Rust crates implement this design: `bimetal` is the runtime that applications depend on, `bimetal-macros` contains the derives and the `export` attribute, `bimetal-ir` defines the contract and the naming rules, and `bimetal-cli` is the command. The command depends on `bimetal-ir` and never on `bimetal`, because a command that linked the runtime would also link PyO3 and would then need

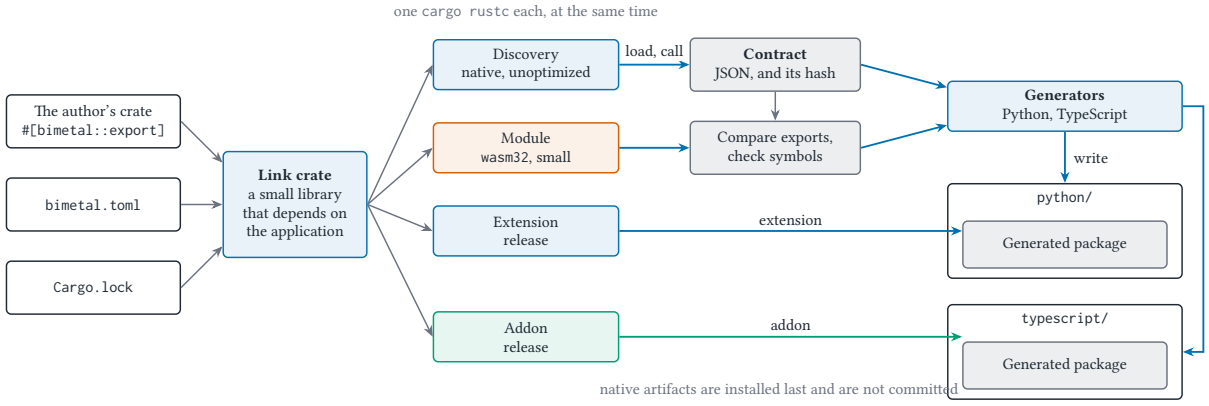


Figure 1: A build. White boxes are the author's files, gray boxes are outputs, and coloured boxes are steps. The discovery build always runs, the extension's build runs when Python is enabled, and the application's choice of JavaScript host selects either the module's build or the addon's.

libpython. For the same reason, the runtime's `python` feature is off except in the extension's build, so an application's own test binaries do not link libpython either.

4 THE CONTRACT

4.1 Reading the Contract from Compiled Code

Each marked item expands to code that registers a description of the item with `inventory` [21]. That crate uses linker constructors, which let one macro expanded in many crates add to a single registry without a central list. The discovery build exists to run this registration and to return its result.

To read the contract, the command loads the discovery build as a library and calls a function named `bimetal_discovery_v1_contract__{link package}`. The function returns a status and a C string, which holds the contract as JSON, a validation error or a panic's message. It catches panics, because a panic that unwound into the command through a C interface would abort the process. Once the command has read the string, it passes the string to a matching `free` function, which releases it. Appendix A gives the format of the JSON.

The discovery build uses the release profile, with optimization off, incremental compilation on and debug information off. These settings keep it cheap to rebuild, and the release profile gives it the same `cfg(debug_assertions)` as the extension. The extra build costs time, and in exchange it provides three properties. First, the contract describes what the compiler built, including the effects of `cfg` attributes, macros and build scripts. Second, the macros reject a type that the contract cannot represent during the build, with an error that names the Rust item. Third, a macro that generates exports needs no special support, because the discovery build runs after macro expansion.

4.2 Agreement with What Was Compiled

The contract comes from one build for the current machine, but a package runs an artifact from another build: a module, an extension or an addon. Four checks make each artifact agree with the contract.

C1, native hosts. The extension and the addon are built from the same sources and for the same target as the discovery build, with the runtime's host feature and a different profile. Each artifact computes a hash of its contract when it loads, and the generated code compares this hash with its own. A mismatch raises an error that says to rebuild.

C2, the module's exports. The module is built for a different compilation target. After this build, the command reads the module's export section. The command stops if the module lacks a function, resource, factory or method of the contract, or has a function or resource that the contract lacks.

C3, unresolved symbols. If the module needs a symbol that nothing defines, the command stops and names the symbol. A C library compiled for the wrong target usually causes this failure.

C4, conditional exports. A `#[cfg]` attribute on an exported method may test only features, joined by `all`, `any` and `not`, because every build compiles the default features of the application's packages, while the module is built for another target. A test of the target is a compile error, which says to put the condition inside the method's body. A `cfg` that a `cfg_attr` adds, at any depth, is also a compile error, because rustc would remove the method after the macro had listed it, and so is an empty `any()`, which holds in no build. A gated method is therefore in every artifact or in none, and C1 and C2 check that this holds. Rust evaluates a `#[cfg]` on a function or an `impl` block before the macro runs, so there C2 alone finds a difference between the targets, as it does for a feature that only a dependency of one target turns on.

Claim 1 (Agreement). *Under A3, if* `bimetal build` *succeeds, each generated call names a function that exists in its host's compiled artifact, with the signature that the generated code expects.*

Argument. For the native hosts, C1 gives the same sources and target as the discovery build, and the hash comparison rejects an

artifact whose contract differs. For the module, C2 gives the same names. The types behind one name come from one Rust source, and by A3, no target changes those types. □

The argument depends on A3 because the command cannot detect a difference inside an export between targets. For example, a constant whose value depends on the target takes the current machine's value in every package. The default of a field is read from the compiled library in the same way, and a package builds its contract again when it loads and refuses to load if that contract differs from the one that the build read. A `Default`, or the function that `#[serde(default = "path")]` names, must therefore give the same value on every call, and the build fails when two calls differ.

4.3 Names

A signature means the same thing on every host, but its names have to follow each language's conventions. The naming rules live in `bimetal-ir`, and the macros and both generators share that one implementation. TypeScript functions, methods and parameters use lower camel case, while Python names stay in snake case. When a language cannot use a parameter's name, the generator renames the parameter in that language only. For example, `class` becomes `class_` in TypeScript and `class_value` in Python.

The command refuses a name that it cannot make safe, and it reports all such names in one error. It refuses an export named `then`, for example, because that export would make a TypeScript module a thenable, and `await import(...)` would then call the export. It also refuses names that the generated package defines itself, and names that collide after renaming. The checks of a language apply only when that language's host is enabled.

5 ONE WRAPPER PLAN

A *wrapper* is the native function that a host calls. One function plans the wrapper for every host, so each feature has one implementation rather than three. The macros check each exported signature once, for the contract and for every host. The planned wrapper then decodes the arguments, obtains the resource, calls the author's function and converts the result. This plan is the same for synchronous and asynchronous calls, and for free functions and methods. Where hosts differ, the planner calls an interface named `HostBackend`, and it never branches on the host itself.

Figure 2 follows one call to `greet` from the generated code to the author's function and back. The generated code runs first: in Python it validates a dictionary as the model and then dumps the model, and in TypeScript it checks the outermost type of each argument. The wrapper then decodes each value once and checks its field rules before it calls the author's function. On the way back, the wrapper checks and encodes the result, and the generated code validates it in Python or freezes it in TypeScript. Section 12 reports what the generated code costs compared with the wrapper alone.

A wrapper for one host compiles only in that host's build, and the application needs no Cargo feature for this. Macro output for the extension passes through `bimetal::__python_items!`, which expands its input only when the runtime has its `python` feature, and `__node_items!` does the same for the addon.

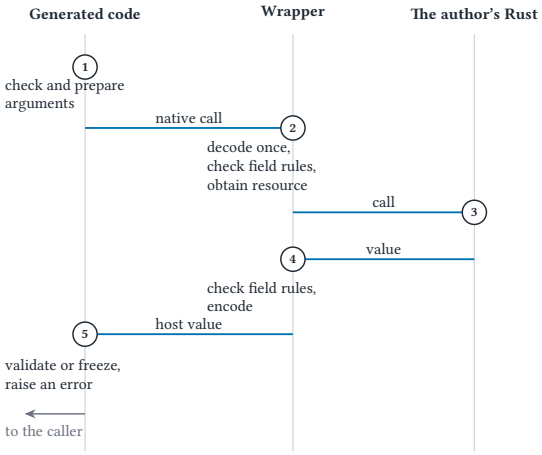


Figure 2: The life of a call. Steps 1 and 5 are generated Python or JavaScript, and steps 2 and 4 are the wrapper.

Table 2: What each host does with a value. A copy here is a copy of the whole payload, and an IPC stream counts as one more copy. Rust's own allocations, such as a `Vec` that a function returns, are not counted.

Value	Extension	Module and addon
Model	Converted by `pythonize`, validated by Pydantic	Converted by `serde-wasm-bindgen` or by the addon's own converter
Bytes	One copy each way	One copy each way
Buffer	One copy each way	One copy each way
Arrow	No copy	An IPC stream and one copy each way
Arrow stream	No copy, one batch at a time	An IPC stream and one copy each way, one batch at a time

A *protocol* is a language feature that a method implements, such as Python's `len()` or iteration with `for`. An author requests one with an attribute such as `#[bimetal(len)]`. A protocol adds no native code: the macro records it in the contract, a validator checks the method's signature, and the generators write the special members that call the method. For example, `len(value)` in Python and `value.length` in TypeScript are generated code around one wrapper.

6 VALUES

How a value crosses depends on how the author writes the signature. Table 2 lists the kinds of value, and this section discusses them in three groups: values that are converted field by field, values that cross as one block of memory, and Arrow data.

6.1 Converted Values

Models, strings, numbers, lists and maps are converted field by field. Serde reads and writes these values through `pythonize` on

the extension, through `serde-wasm-bindgen` on the module, and through a serializer and a deserializer of our own on the addon. Our deserializer follows `serde-wasm-bindgen`'s checks, order and error messages, apart from a limit on nesting. In Python, a scalar or a container of scalars passes unchanged, and so does an optional argument that is absent. Pydantic writes every other argument in one pass, in its JSON mode, where a model is a dictionary and a date is ISO 8601 text. When a value returns, Pydantic validates it, unless it is a Boolean, an integer, a float or a string, which the extension already makes as Python holds it.

Rust checks an author's field rules on every host. It checks every argument and result of an export, and what a callback returns to Rust, but not the arguments that Rust passes to a callback. The check may look redundant in Python, where Pydantic enforces the same rules. However, `model_copy(update=...)` skips Pydantic, and TypeScript has no equivalent of Pydantic at all. A returned value that breaks a rule indicates a bug in the Rust code, and the caller then receives an untyped error. A field that is absent holds the Rust `Default` of its type or what its default function returns, which no caller passed. The model's registration calls that function when the contract is read and checks the value against the field's own rules and the rules inside it, so a default that breaks a rule fails the build. The registration also fails the build when the value holds a NaN or an infinity at any depth, because JSON writes either as `null`, which inside an `Option` reads back as `None`. For the same reason, the derive rejects `skip_serializing_if` on a field with a default function, because Serde would leave out a `None` and then read the absent field as what the function returns. The derive also checks the `Default` of a scalar or a list against the field's rules, because it knows that value before the library is compiled.

6.2 Bytes and Buffers

Bytes and buffers cross as one block of memory, which avoids the cost of a host object for each element. A plain `Vec<u8>` crosses as a list of integers unless the author writes `#[bimetal(bytes)]`, which passes it as Python `bytes` or a `Uint8Array`. A *buffer* is the same idea for a numeric array, such as a slice of `f64`, which reaches the caller as a NumPy array or a typed array. A returned buffer arrives in Python as a `bytearray`, which NumPy wraps without a further copy.

6.3 Arrow Values and Streams

Arrow values show the largest difference between hosts, because only the extension can share their memory with Rust. On the extension, arrays, batches, tables and schemas cross through the PyCapsule interface [2], which is built on the C Data Interface [1], without a copy in either direction. The runtime accepts any producer of this interface, after it validates each incoming value. Our tests pass pyarrow values, and one binding crate's test passes a Polars DataFrame. The other two hosts cannot share memory in this way. On the module, a view into WebAssembly memory stops working when the memory grows, and on the addon, the apache-arrow JavaScript library has no C Data Interface. Arrow values on these two hosts therefore cross as an IPC stream, with one copy into Rust's memory and one copy out.

A *stream* of record batches crosses one batch at a time. An export can return a `RecordBatchStream`, which wraps an asynchronous Rust stream or a blocking reader. Python receives it as a pyarrow `RecordBatchReader`, which is built on the C Stream Interface [1], and any thread can ask this reader for the next batch, including the worker threads of a query engine such as DuckDB. A parameter can also take a stream, from any producer of the interface, and the runtime then reads each batch only when the Rust code asks for it. A synchronous export reads on its caller's thread, while an asynchronous export, which starts on the runtime because it takes a stream, reads each batch on a blocking thread of the runtime and never on a worker. Dropping a stream drops its source, which cancels the source's work. The first error ends a returned stream: every later read fails with the same message, and a typed error keeps its code at the start of that message.

On the module and the addon, TypeScript reads a returned stream as an asynchronous iterator, and each batch crosses as its own IPC stream. In the other direction, a JavaScript source writes a batch only when Rust asks for one. JavaScript makes these batches asynchronously, and a synchronous call on its one thread could not wait for them. The command therefore refuses a stream parameter of a synchronous export when an application builds a TypeScript package.

Claim 2 (Streams without the lock). *Rust never waits for a batch of a stream parameter while it holds the interpreter lock. A returned stream releases the lock before it waits for a batch.*

Argument. A synchronous export runs without the lock, so it reads its parameter's batches without the lock. An asynchronous export with a stream parameter runs on the runtime's workers, which never hold the lock, and it reads each batch on a blocking thread. A read of a returned stream checks whether its thread holds the lock. If the thread holds it, the read releases it, waits for the batch and takes the lock back. □

This claim matters because a source may need the lock to make its next batch, as a pyarrow reader over a Python generator does, or a stream that calls a Python function. If a read waited with the lock held, the source would wait for the read, and neither would finish. One test covers this case: a Python function feeds a returned stream, and an import that holds the lock reads the stream. Without the release, that test does not finish.

Two further rules apply to Arrow. First, streams do not survive a fork. Once the runtime has started, a forked child cannot read a stream that its parent returned: every read fails in the child, and the child never drops the stream, because the stream may hold tasks of the parent's runtime. Second, an application's Arrow values must have the same Rust types as the Arrow crate that the application already uses, and Rust treats two versions of one crate as different types. The runtime therefore has an `arrow` feature for arrow-rs 60 and an `arrow-59` feature for arrow-rs 59. Each feature names its crates the same way at the crate root, which gives the runtime one code path for both versions. A version range would not work, because Cargo keeps both versions and gives each dependent the highest one it allows.

7 RESOURCES

A resource must keep three of Rust's guarantees after a caller in another language holds a reference to it: no reader sees a value during a write, no call uses a freed value, and a method that takes `&mut self` has exclusive access. To provide these guarantees, every resource keeps its value in one structure named `Resource`, which holds a shared cell around an `Arc` of a Tokio read-write lock. Seven rules govern this structure.

- **R1, obtaining.** A call clones the `Arc` when it starts. This means that a call keeps the value alive even after the resource is closed (R3), and Rust drops the value after the last call ends.
- **R2, locking.** A `&self` method takes the read lock, and a `&mut self` method takes the write lock. Each keeps its lock until its result has been converted.
- **R3, closing.** `close()` empties the shared cell without waiting for running calls. If no call has a clone, Rust drops the value inside `close()`, on the calling thread. A later call fails with an error that names the method.
- **R4, taking.** A `self` method or a parameter of type `T` moves the value out and closes the resource. This succeeds only if no other call has a clone. If a call fails after it took a `T` argument, the resource stays closed and its value is gone.
- **R5, one use in a call.** If a call receives one resource twice, in two parameters or as its receiver and a parameter, the call fails.
- **R6, thread safety.** When Python is enabled, a resource's type must be `Send` and `Sync`. The compiler checks this bound when it builds the extension.
- **R7, inside a callback.** While a thread runs a function that a `bimetal::Callback` called, a synchronous call on that thread does not wait for a lock. If the lock is taken, the call fails with an error that says the resource is busy.

Claim 3 (Aliasing). *A caller cannot use a freed resource. While one call has* `&mut T`*, no other call has any reference to that* `T`*.*

Argument. By R1, a call obtains the value only through a clone of the `Arc`, and that clone keeps the value alive. By R3 and R4, a call that starts after a close or a move finds the cell empty. By R2, the read-write lock makes a `&mut T` exclusive. By R4, a move succeeds only when no other call has a clone. `Resource` and the code that the macros emit use no `unsafe` code. On the addon, the runtime reads the object behind a method's receiver or an argument through `napi_unwrap` and one unchecked dereference, after a type-tag check. Rust's type system guarantees the remaining cases [12]. □

Claim 4 (No waiting for itself). *A call cannot wait for itself through its receiver, its arguments or a callback.*

Argument. A call could wait for itself in two ways. First, it could take one resource's lock twice. By R5, such a call fails before it takes the second lock. Second, a function that the call runs through a callback could make a synchronous call that needs a lock the outer call holds. By R7, this inner call fails and does not wait. The function then raises an exception, and the outer call receives it as an error value. □

Table 3: What a call does when its resource is busy. A `self` method needs the value alone, and it fails if any other call has a clone.

The call	Python	JavaScript
Sync method	Waits	Throws an error
Sync, in a callback	Fails with an error	Throws an error
Async method	Waits in its future	Waits in its future
`self` method	Fails with an error	Fails with an error

R7 is what makes this argument hold on the extension, the one host where a synchronous call can wait. Suppose a synchronous `&self` method a calls a Python function, and that function calls a `&mut self` method b of the same resource. Without R7, b would wait for a's read lock while a waited for the function to return, and neither would finish. With R7, b fails at once, although a read inside the same function succeeds, because readers share the lock. The rule has a cost: if the function calls a different resource while another thread is using it, that call also fails, as a synchronous call in JavaScript does. The rule cannot be narrower, because a thread cannot tell which call holds a lock.

Two cases fall outside Claim 4. In the first, call a takes resource x and then waits for y, while call b takes y and then waits for x. These two calls wait for each other, as the holders of any two locks can. Bimetal does not order locks globally, because an author's Rust code would then have to know about the order. In the second, the function that a callback runs starts a thread of its own and waits for it. R7 applies only to the thread that runs the function, so a call on the new thread can still wait for the outer call.

7.1 What a Host Does with a Busy Resource

A resource is *busy* when a call needs a lock that conflicts with one that another call holds. The hosts cannot all offer the same behaviour for a busy resource, and Table 3 lists what each one does. To see why they differ, suppose a caller starts an asynchronous call a to a `&mut self` method, and a takes the write lock. Before a finishes, the caller makes a synchronous call b to the same resource on the same thread. In JavaScript, b throws an error, because the code that would finish a could never run while b waited on the one thread. In Python, b waits and releases the interpreter lock, so other Python threads keep running. Call a finishes on a Tokio thread, and b then continues. If b runs on the event loop's own thread, however, it blocks that loop while it waits.

The hosts also differ in when an asynchronous method takes its lock. On the JavaScript hosts, the method tries its lock on the calling thread when the caller makes the call. In Python, this attempt happens when the generated coroutine first runs. A call that gets the lock holds the resource from that moment, even on the module, where the Rust body starts in a later microtask. If the lock is taken, the call waits in its future and joins the lock's queue when its future is first polled. On the native hosts, a multi-threaded runtime makes every poll after the first, so calls that find the lock taken have no guaranteed order among themselves.

7.2 Blocks and Failure

Python's `with` and TypeScript's `using` [5] close a resource when a block ends. If a method is marked `exit`, it runs first and receives a flag that says whether the block failed. Python passes the real exception state, but JavaScript does not tell the dispose method whether the block threw, so the generated TypeScript always passes `failed = true`. The consequence shows in a resource whose `exit` method commits when `failed` is false and rolls back otherwise: a TypeScript author must commit explicitly before the block ends. If the author forgets, the resource rolls back, which is the safer of the two outcomes.

7.3 Callbacks

A parameter of type `bimetal::Callback<A, R>` holds a Python or JavaScript function that Rust can keep and call from any thread. What happens during that call depends on the host. On the extension, the call takes the interpreter lock, and on the module it runs on the only thread. On the addon, a call on the JavaScript thread runs directly, while a call from another thread hands the work to the JavaScript thread and waits. This last case has a hazard. A synchronous export occupies the JavaScript thread while it runs, so it must not wait for a callback made on another thread, because that callback needs the same thread. The runtime cannot detect this case. On every host, if the function raises an exception, Rust receives an error value, which the author maps into a typed error.

8 FAILURES

A call can fail with a typed error, a bad argument, a panic or a trap, and only the first of these is part of the API. Each Rust error type becomes a class with a `code` field, and the declared type of that field lists every code, so a type checker catches a misspelled one. A bad argument raises the language's own error, which names the function and the parameter. Panics and traps are bugs in the Rust code, and the rest of this section concerns them.

A panic must not unwind into a caller, because that would abort the process. On the native hosts, the wrapper catches the panic and reports it. The extension raises `PanicException`, which derives from `BaseException` so that `except Exception` does not hide a bug, and the addon throws an `Error`. Both hosts stay usable afterwards. A later call can, however, reach a resource whose `&mut self` method panicked partway, because Tokio's lock does not record a panic. So that a panic never ends the process, the command builds these artifacts to unwind even when the application's Cargo profile sets `panic = "abort"`.

The module cannot catch a panic, because in WebAssembly a panic becomes a trap. The trap leaves the module with its memory, locks and asynchronous executor as they were partway through a call. It also leaves every other pending promise waiting for a future that can never finish, so an application would hang with no error. Bimetal therefore wraps every export of the module in a function called the *guard*, which turns a trap into errors. The command changes the code that wasm-bindgen generates, so that every call into the module passes through the guard, including the asynchronous executor's microtasks and the finalizers.

Figure 3 shows what happens when a call traps while another call is pending. Suppose a caller starts an asynchronous call *a*. As

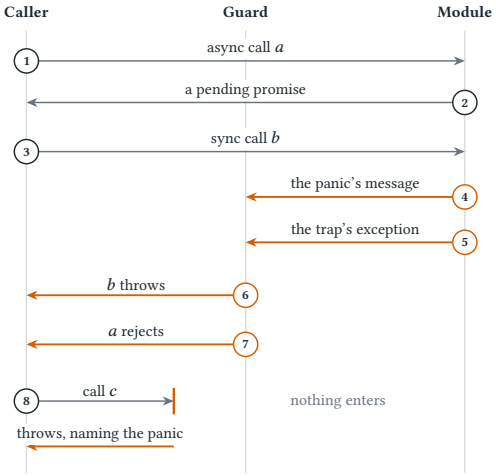


Figure 3: A trap, read from top to bottom. Without the guard, call *a*'s promise in step 7 would stay pending for ever, and call *c* in step 8 would enter a module in an unknown state.

every asynchronous wrapper does when its call starts, the wrapper for *a* registers the function that rejects *a*'s promise. The caller then makes a synchronous call *b*, and *b* panics. The panic hook gives the guard the panic's message, the module traps, and an exception leaves the module. The guard treats any exception from the module as proof of a trap, because Rust returns its errors as values and never throws. It records the abort and rejects *a*'s promise, and call *b* throws an `Error` with the panic's message. When the caller later makes a call *c*, the generated code finds the record and throws at once, without entering the module. From this point `close()` does nothing, so that a `using` block reports the panic and not a second failure. The process keeps running, and only a reload makes the module callable again.

Claim 5 (Trap). *After a trap, no call enters the module, and no promise that depends on the module stays pending.*

Argument. Every path into the module passes through the guard. When the guard sees a trap, it records the abort and rejects the pending promises. It then rethrows to a wrapper's call, or returns from an outermost microtask or finalizer, whose exception no caller could catch. The generated code checks the record before it enters the module, and the guard checks it again. Each asynchronous wrapper registers its promise's rejection when its call starts, and the guard rejects all of them. A callback can call back into the module, and that inner call can trap. The callback then fails, Rust receives this failure as an error value, and the outer call could return normally from an unusable module. The guard therefore throws when the outermost call returns. □

9 ASYNC

An `async fn` becomes a Python coroutine and a TypeScript function that returns a `Promise`, and the host decides what runs it. The native hosts run these futures on a Tokio runtime that the process starts on demand, while the module runs them on the JavaScript event loop.

A native host polls a future once on the caller's thread before it gives the future to the runtime, because a call that finishes without a wait, such as a read from memory, then needs no worker. Such a call settles its asyncio future or its promise at once, while a call that waits becomes a task of the runtime, whose first poll replaces the waker of the caller's poll. That first waker wakes no one, so a future must register its waker on every poll, and code before the first wait runs outside any Tokio task. The extension makes that first poll without the interpreter lock, as it runs a synchronous export, so long work before the first wait stops the caller's event loop but no other Python thread.

A call that takes a `Callback` or a `RecordBatchStream` is the exception, and a worker makes its first poll, because such a call may wait for the caller's event loop before its first wait. A Python function may wait for a coroutine on that loop, and on the addon JavaScript writes a stream argument on the thread that runs the loop. On the caller's thread, either wait would never end. The macros know the type of each parameter, so they choose where each asynchronous export starts when they expand it.

On the extension, a worker thread must be able to finish a call without waiting for the interpreter lock. References to the event loop and to the asyncio future are therefore kept in a structure that only code holding that lock can touch. A `Callback` argument is the exception, because the task owns its Python function, and if the last clone drops on a worker thread, PyO3 defers the release to its reference pool. When the task finishes, a thread that is allowed to block takes the lock and converts the result. Cancellation works in Python only. When a caller cancels the future, a callback aborts the task, and Tokio drops the Rust future at its next opportunity. A JavaScript promise cannot be cancelled, so a call there runs to its end.

Some synchronous calls also run inside the Tokio runtime: constructors, factories, and functions and methods that return a resource, because a resource may need to spawn tasks that it keeps. Other synchronous calls do not run inside it, and a package with no resources or asynchronous calls never starts a thread. A child process created by `fork` starts its own runtime, because the parent's worker threads do not exist in the child. Calls that were running in the parent do not continue in the child, and cancelling one there does nothing. If one of those calls held a resource, the resource stays held in the child, and a call there that needs it waits for ever.

The module has no Tokio runtime, so an author whose code uses Tokio's I/O, timers or `spawn` needs another implementation for that target. A library that spawns tasks internally, as DataFusion does, can still run on a single-threaded runtime of its own through `block_on`, as long as nothing it awaits needs I/O or a timer.

10 REPRODUCIBLE OUTPUT

An application commits its module and generated source, and `bimetal check --rebuild` compiles them again and compares the bytes. This comparison is useful only if two machines produce the same bytes [11]. By default they do not, because absolute paths reach the output in two ways.

The first way is through panic messages, which name source files and are part of the module. The command therefore remaps every local path in every compilation for a build's target. The workspace's files become relative paths, the Cargo home becomes `/cargo`, the target directory becomes `/target`, and the link crates become `/bimetal-link`. Packages outside the workspace become `/source/{name}`, and the standard library's sources become `/rustc/{commit}`. Diagnostics map these paths back, so an author still sees the real ones.

The second way is through each crate's identity, which comes from rustc's `-C metadata` flag. This identity appears in symbol names, decides the order of the module's functions and is part of every `TypeId`. Cargo hashes into this flag the absolute path of each local package that lies outside the workspace being built. The link crate lies outside the application's workspace, so the application's packages are among these local packages. For build scripts and procedural macros, Cargo also hashes in the build machine's target triple. The command therefore runs rustc through a small program of its own, which replaces the flag with a hash of the package's remapped directory, the crate's name and crate types, and its `--cfg` flags.

Lastly, the command strips the module's `name` and `producers` custom sections, which contain debug names and toolchain metadata.

Claim 6 (Reproducibility). *Under A1 and A2, two checkouts of one commit, at any path, on Linux or macOS, with one toolchain, produce the same module and the same generated sources.*

We support this claim with tests rather than a proof. Continuous integration rebuilds the committed examples on Linux and macOS, on machines that did not build them, and compares the bytes. The claim excludes Windows, because rustc keeps Windows separators in the part of a path after the remapped prefix, and those separators reach the module through panic messages. A committed module should therefore come from another platform.

10.1 A Fingerprint That Stays Equal

To avoid compiling when nothing changed, the command records a fingerprint of a build's inputs. This fingerprint is committed, so it must not differ between machines: if it covered anything specific to one machine, a build on another machine would change a committed file. The command therefore divides what a build read into two groups (Figure 4). The inputs that every checkout has are hashed into the *source fingerprint* in `bimetal.toml`. The command assumes that every checkout of a commit has the files among them, which are the files that `.gitignore` does not exclude. Every other input is a *local input*, and the command records these in an ignored file.

The generator itself is one of the inputs in the first group. The command's build script hashes the generator's own sources into the source fingerprint, so a change to the generator rebuilds applications even when the version number stays the same. If an application has no module, the command records what the discovery build read instead. That list is the same on every machine only while no code compiled for one operating system alone reads a file. A1 applies here too, because the command cannot detect what a build script reads without telling Cargo.

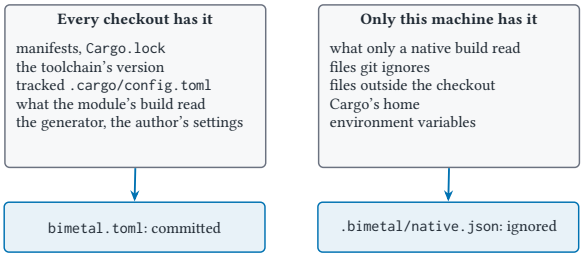


Figure 4: Where a build's inputs are recorded. The module's build reads the same files on every machine, and its list of inputs is therefore the same on each machine.

10.2 Ownership of Files

A generator that writes into a project must not destroy an author's work. The command therefore records every file that it writes, and it writes only inside the package folder. Before it changes anything, it stops if a new path would overwrite a file that it does not own. It then copies each changed file through a temporary file beside the destination, deletes its own stale files, and updates `bimetal.toml` last. If a build stops midway, the project contains only the command's files beside the author's, and the next build adopts them. A file whose bytes did not change keeps its modification time, so a file watcher does not run again.

11 TESTING

Bimetal's promises concern what a caller sees, so most of its tests build real packages and call them from real runtimes. An example application called `showcase` uses every feature except `arrow-59`, and its tests run on all three hosts. A second example returns and takes streams, and its tests read and pass them with pyarrow, DuckDB and Polars. Every test file runs on the addon unless a short list marks it as specific to the module, so a new file is tested on the addon by default. The Python tests also run `mypy --strict` and strict pyright on the generated package, and the TypeScript tests type-check against the generated declarations. Finally, each package is built as a wheel or a tarball, installed into a new project outside the repository, and loaded there.

Below this level, unit tests cover the naming rules, the fingerprints and the writing of files, and a set of compile-fail tests checks the macros' error messages. The runtime's own tests embed CPython to drive asynchronous calls with asyncio and to test `fork`. A further test requires every Rust, Python and TypeScript sample in the documentation to appear in a program that runs.

To learn what real code needs, we also wrote binding crates for five large Rust projects: DataFusion, Oxc, Polars, Tantivy and Typst, which range from about 134,000 to about 575,000 lines of Rust. Each binding crate is a few hundred lines. We wrote each API as a user of that project would want it, before we checked what Bimetal allowed. This work led to most of the features in Section 7, including resources returned from methods, callbacks and `exit`.

Table 4: Median time per call into the `showcase` example. "Wrapper only" calls the wrapper directly, without the generated code, and we measured this for models only in Python. An Arrow call passes a batch of 16 MB and receives a table of one batch. The two asynchronous rows come from a later run of the per-call step, which started at a load average of 22 for the extension and 5 for the addon, and the module did not change between the runs. The module has no figure for a call that does not wait, because we did not measure it under comparable load.

Call	Extension	Module	Addon
No arguments	53 ns	28 ns	19 ns
Wrapper only	35 ns	28 ns	18 ns
10,000 models in	53.4 ms	45.1 ms	51.5 ms
Wrapper only	16.3 ms		
10,000 models out	4.1 ms	2.6 ms	3.5 ms
Wrapper only	0.6 ms		
1 MiB of bytes	44 μs	324 μs	72 μs
1,000,000 Arrow rows	4.8 μs	2.1 ms	2.0 ms
Async, one yield	34.4 μs	0.7 μs	12.5 μs
Async, no wait	0.37 μs		0.33 μs

12 WHAT IT COSTS

Bimetal has three kinds of cost: time on every call, memory for copies, and time to build. Table 4 gives measurements of the first, from two runs on an Apple M5 laptop with 10 cores and 32 GiB of memory, on macOS 27, with Rust 1.94, Python 3.11 and Node.js 24. Each cell is the median of at least 30 samples, and the caption says which run each row comes from. These numbers show where the costs are, and they do not compare Bimetal with other tools.

The table shows three things. First, most of the cost of a Python call with models is the generated code. Of 53.4 ms for 10,000 models that contain dates and JSON, 16.3 ms is the wrapper, and the rest is the generated code, which validates the list and writes each model for the wrapper. Second, an Arrow call on the extension took the same time at every batch size that we tried: 4.5 to 4.9 μs with two fixed-width columns, from 1,000 to 4,000,000 rows. The time on the JavaScript hosts grew with the batch. Third, an asynchronous call that waits once cost more on the native hosts than on the module, while a call that does not wait cost 0.37 μs on the extension and 0.33 μs on the addon. The two calls differ only in one yield, which moves the call to a worker thread and back, so that yield costs about 34 μs on the extension and 12 μs on the addon.

We also measured peak memory, to check the copies in Table 2. When a call reversed 100 MiB of bytes, peak memory rose by 3.0 times the payload on every host: one copy in, Rust's result and one copy out. When an Arrow call passed 16 MB, peak memory rose by 0.06 times the payload on the extension and by about 4.1 times the payload on the JavaScript hosts, which is one stream and one copy in each direction.

Build time is the third cost. A first build compiles every dependency once for each host, which took 14 to 42 seconds for three of our small examples and several minutes for each of the five large projects. A build where nothing changed took 70 to 88 milliseconds, and `bimetal check` took about the same.

Table 5: Tools closest to Bimetal.

Tool	Runs in	Definition	Types for callers
Bimetal	Python, Node.js, browsers	Compiled Rust	Generated
PyO3 [18], maturin [17]	Python	Rust macros	By hand by default
napi-rs [14]	Node.js	Rust macros	Generated
wasm-bindgen [23]	Browsers, Node.js	Rust macros	Generated
UniFFI [13]	Kotlin, Swift, Python, Ruby, others	Macros or a file	Generated
Diplomat [4]	C, C++, Dart, JavaScript, .NET, Kotlin, Python	A bridge module	Generated

13 LIMITATIONS

The arguments in this paper are informal. The claims rest on the code and on the tests that exercise it. There is no machine-checked model, and the claims do not cover bugs in PyO3, wasm-bindgen, Node-API or the compiler.

Three interfaces need `unsafe` *code.* These are Node-API's calls, the C Data Interface and the discovery interface. Tests and code review check this code, and the tests do not run under a sanitizer or Miri.

Generated code costs time on every call. The generated Python validates models in both directions, whether or not a caller needs it, and a caller cannot turn this off.

The contract has gaps. The command does not enforce A3 inside an export. Bimetal does not support untagged or adjacently tagged enums, tuple variants that hold other than one value, `#[serde(flatten)]`, generic types, `u128`, maps with keys other than strings, bytes inside a model, keyword-only arguments or asynchronous callbacks. In TypeScript, only an asynchronous export can take a stream of record batches.

Each host costs its own build. When a project is large, a first build takes minutes and gigabytes, because every dependency compiles once for each host.

14 RELATED WORK

Table 5 compares Bimetal with the closest tools, as their own documentation described them on 6 November 2026. PyO3, napi-rs and wasm-bindgen each give an author full control of one runtime, and Bimetal uses PyO3, wasm-bindgen and napi-rs's low-level `napi-sys` crate underneath. These tools trade repeated work for a result shaped exactly to one runtime. UniFFI and Diplomat generate bindings for many languages from one definition, and they cover far more languages than Bimetal does. In exchange, Bimetal offers Pydantic models, Arrow without a copy in Python, and `async` and protocols in both of its languages.

Earlier generators read a definition that is separate from the compiled code. SWIG [3] generates bindings from C and C++ headers, and Thrift [20] generates them from an interface definition language. Bimetal reads the definition from compiled code instead, so the definition matches what the compiler built for the discovery build, and C1 to C4 and A3 cover the other builds. Other work checks or removes the boundary rather than generating it. Furr and Foster check foreign calls statically against the host language's types [6], and Jeannie lets a program mix Java and C in one file and generates the boundary [9]. Grimmer et al. remove the boundary inside one multi-language runtime [7], which is not possible when the caller runs on CPython or in a browser.

Two further lines of work bear on this paper. WebAssembly's own design explains several of the limits described here [8]: a trap has no recovery, and memory growth detaches views. Jangda et al. compared WebAssembly with native code on the SPEC CPU benchmarks [10], whereas we report what the boundary costs and not what running inside the sandbox costs. Lamb and Zacchiroli describe the Reproducible Builds effort [11], and our rules are one specific case of it, which we needed to compare a committed binary in continuous integration.

15 CONCLUSION

Bimetal treats a compiled Rust library as the definition of its API, and generators write the Python and TypeScript packages from that definition. Reading the contract from a build ties the generated code to what the compiler built, and four checks cover the other builds. Committing the generated code lets continuous integration compare it with a fresh build, byte for byte. One planner gives the three hosts one implementation of each feature. Where hosts cannot offer the same behaviour, as with blocking, cancellation and panics, each package follows its host, and each failure says what happened. In exchange, a call costs more than a hand-written one, and each host costs a build.

A THE CONTRACT'S FORMAT

The discovery function returns one JSON object, whose keys are in camel case. A program that reads this format can generate a package for another language.

```
{"packageName": s, "packageVersion": s,
 "types": [TypeDef], "errors": [ErrorDef],
 "functions": [FunctionDef], "resources": [ResourceDef],
 "constants": [ConstantDef]}
```

A `TypeDef` has an owner package, a Rust module, an id, a name, documentation and a shape. The shape is an object tagged by `kind`: a struct with fields, a string enum, a tagged enum, an external enum or an alias. A variant of an external enum carries nothing, one value or fields, under its name. A field has its Rust name, its name in JSON, documentation, a type, whether it is required, a default, and constraints: `literal`, `minLength`, `maxLength`, `ge` and `le`.

A `TypeRef` is an object tagged by `kind`, whose value is one of `unit`, `bool`, `int`, `float`, `string`, `dateTime`, `date`, `duration`, `json`, `option`, `list`, `map`, `tuple`, `named`, `bytes`, `fixedBytes`, `buffer`, `arrow`, `resource` and `callback`.

A `CallableDef` has a Rust name, a name in the language, documentation, parameters, an optional error and whether the call is asynchronous. A parameter has its names, its type and an optional default. A `FunctionDef` contains its owner, its module, its native name, a `callable` and a return type. A `ResourceDef` contains its owner, its module, an id, a name, a native name, documentation, an optional constructor, factories and methods. A method contains a `callable`, a return type and an optional protocol.

An `ErrorDef` lists codes, and each code has documentation and an optional definition of its details. A native name such as `__bimetal_function_greeter_fb54ba6c799a7247` is globally

unique, and the generated code looks up this name in the compiled artifact.

The contract hash is FNV-1a over the discovery JSON. The extension exposes this hash as `__bimetal_contract__`, and the addon exposes it to its loader in the same way.

REFERENCES

[1] Apache Arrow Project. 2026. The Arrow C Data Interface. https://arrow.apache.org/docs/format/CDataInterface.html. Accessed 6 November 2026.

[2] Apache Arrow Project. 2026. The Arrow PyCapsule Interface. https://arrow.apache.org/docs/format/CDataInterface/PyCapsuleInterface.html. Accessed 6 November 2026.

[3] David M. Beazley. 1996. SWIG: An Easy to Use Tool for Integrating Scripting Languages with C and C++. In *Proceedings of the 4th USENIX Tcl/Tk Workshop*. USENIX Association.

[4] Diplomat Project. 2026. Diplomat: Generating FFI Definitions for Rust. https://github.com/rust-diplomat/diplomat. Accessed 6 November 2026.

[5] Ecma TC39. 2026. ECMAScript Explicit Resource Management. TC39 proposal, https://github.com/tc39/proposal-explicit-resource-management. Accessed 6 November 2026.

[6] Michael Furr and Jeffrey S. Foster. 2005. Checking Type Safety of Foreign Function Calls. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '05)*. ACM, 62–72. https://doi.org/10.1145/1065010.1065019

[7] Matthias Grimmer, Chris Seaton, Roland Schatz, Thomas Würthinger, and Hanspeter Mössenböck. 2015. High-Performance Cross-Language Interoperability in a Multi-Language Runtime. In *Proceedings of the 11th Symposium on Dynamic Languages (DLS 2015)*. ACM, 78–90. https://doi.org/10.1145/2816707.2816714

[8] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and J. F. Bastien. 2017. Bringing the Web up to Speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '17)*. ACM, 185–200. https://doi.org/10.1145/3062341.3062363

[9] Martin Hirzel and Robert Grimm. 2007. Jeannie: Granting Java Native Interface Developers Their Wishes. In *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '07)*. ACM, 19–38. https://doi.org/10.1145/1297027.1297030

[10] Abhinav Jangda, Bobby Powers, Emery D. Berger, and Arjun Guha. 2019. Not So Fast: Analyzing the Performance of WebAssembly vs. Native Code. In *2019 USENIX Annual Technical Conference (USENIX ATC '19)*. USENIX Association, 107–120.

[11] Chris Lamb and Stefano Zacchiroli. 2022. Reproducible Builds: Increasing the Integrity of Software Supply Chains. *IEEE Software* 39, 2 (2022), 62–70. https://doi.org/10.1109/MS.2021.3073045

[12] Nicholas D. Matsakis and Felix S. Klock. 2014. The Rust Language. *ACM SIGAda Ada Letters* 34, 3 (2014), 103–104. https://doi.org/10.1145/2692956.2663188

[13] Mozilla. 2026. UniFFI: A Multi-Language Bindings Generator for Rust. https://github.com/mozilla/uniffi-rs. Accessed 6 November 2026.

[14] napi-rs Project. 2026. napi-rs: A Framework for Building Compiled Node.js Addons in Rust. https://github.com/napi-rs/napi-rs. Accessed 6 November 2026.

[15] Node.js Project. 2026. Node-API. https://nodejs.org/api/n-api.html. Accessed 6 November 2026.

[16] Pydantic Project. 2026. Pydantic. https://docs.pydantic.dev. Accessed 6 November 2026.

[17] PyO3 Project. 2026. Maturin: Build and Publish Crates as Python Packages. https://github.com/PyO3/maturin. Accessed 6 November 2026.

[18] PyO3 Project. 2026. PyO3: Rust Bindings for the Python Interpreter. https://github.com/PyO3/pyo3. Accessed 6 November 2026.

[19] PyO3 Project. 2026. Typing and IDE Hints for Your Python Package. PyO3 user guide, https://github.com/PyO3/pyo3/blob/main/guide/src/python-typing-hints.md. Accessed 6 November 2026.

[20] Mark Slee, Aditya Agarwal, and Marc Kwiatkowski. 2007. *Thrift: Scalable Cross-Language Services Implementation*. Technical Report. Facebook.

[21] David Tolnay. 2026. inventory: Typed Distributed Plugin Registration. https://github.com/dtolnay/inventory. Accessed 6 November 2026.

[22] Martin von Löwis. 2009. PEP 384: Defining a Stable ABI. https://peps.python.org/pep-0384/.

[23] wasm-bindgen Project. 2026. wasm-bindgen: Facilitating High-Level Interactions Between Wasm Modules and JavaScript. https://github.com/wasm-bindgen/wasm-bindgen. Accessed 6 November 2026.